

The Silent Tax: How Typos Reshape Reasoning in Thinking LLMs

Shiran Hamami

207487026

shiranhamami@mail.tau.ac.il

Ron Shtricker

209524552

ronshticker@mail.tau.ac.il

Dolev Abudi

323096099

dolevabudi@mail.tau.ac.il

Ido Azoulay

212710958

idoazoulay1@mail.tau.ac.il

Abstract

Reasoning models perform strongly on clean mathematical and scientific benchmarks, but real users often write with typos. We study how such errors affect not only final-answer accuracy but also the reasoning process itself. We introduce a controlled typo generator that independently varies the number of corrupted words and the share of typos that produce another valid English word (real-word typos). We apply it to 500 questions from each of GSM8K, MATH-500, and ARC-Challenge and evaluate DeepSeek-R1-Distill-Qwen-7B across all corruption settings. Typos consistently reduce accuracy most on GSM8K, from 92.8% on clean questions to 65.1% in the most corrupted setting. At a fixed number of typos, real-word typos are more harmful than non-word ones: about $1.7\times$ per typo on GSM8K and $1.3\times$ on MATH-500, while on ARC-Challenge only real-word typos have a detectable effect. The reasoning traces show that the model often notices that something is wrong: it reasons for longer and expresses more uncertainty, but barely re-checks its work more often. Finally, we introduce three mitigation strategies but none of these simple mitigations (a typo warning, rewriting the question, and spell checking) reliably helps. The main challenge is not detecting corrupted input but recovering its intended meaning.

1 Introduction

Chain-of-thought prompting and reasoning-focused models have substantially improved performance on mathematical and scientific benchmarks (Wei et al., 2022; Guo et al., 2025). However, benchmark questions are carefully written, while real users often make typing mistakes such as swapped letters, missing characters, or nearby-key errors. Strong performance on clean benchmark text may therefore not fully reflect reliability in real-world use. This gap motivates our study.

Prior work shows that typos hurt LLM performance, especially on reasoning tasks (Gan et al., 2024; Zhao et al., 2026), and attributes part of this sensitivity to subword tokenization (Chai et al., 2024). These studies, however, score only final answers and do not analyze what happens inside the reasoning process as a result. Reasoning models make this question especially interesting: their long reasoning traces give them many opportunities to notice a corrupted word, infer what was intended, and recover before producing the final answer. In this study, we also evaluate the reasoning trace itself.

To study this, we created a typo generator that separates two factors: how many words are corrupted and what kind of corruption they contain. In particular, we vary the share of typos that accidentally produce another valid English word. This allows us to investigate and evaluate the difference between real-word and non-word typos while the number of typos stays fixed, a comparison that previous work has not made. We evaluate DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025) across the full set of conditions on three datasets with different reasoning characteristics. Beyond measuring final-answer accuracy, we also examine the reasoning traces themselves using word-level alignment and an LLM judge.

Our main contributions are: (i) a controlled experimental setup that independently varies typo rate and the proportion of real-word typos across three datasets; (ii) an analysis of how typos affect the reasoning process itself, including changes in model uncertainty, whether the model notices corrupted words, and whether it attempts to repair them, and (iii) an evaluation of three simple mitigation strategies: warning the model that the input may contain typos, asking it to correct possible typos before solving the problem, and applying an external spell checker before the model sees the question.

Our main findings are as follows. Typos cost up

Clean (model answers 18, correct) Janet’s ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers’ market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers’ market?
$r=25\%$, $\rho=40\%$ (model answers 24, wrong) Janet’s duck lay 16 eggs per adqy. Sje east rthere for breakfast every morning and bakes muffins for uher friend every azy with four. She sells the remainder at the farmers’ market daily for \$2 per resh duck egg. How much in dollars does she make every day at the faremrs’ market?

Figure 1: A single GSM8K item under our generator. Numbers and LaTeX are protected; only alphabetic words are corrupted. Four of the ten typos are *real words* (eats→east, ducks→duck, friends→friend, fresh→resh); the spell checker accepts the first three. The model answers 24 instead of 18.

to 27.7 percentage points of accuracy on GSM8K, 10.7 on ARC and 8.2 on MATH-500, and the loss comes from wrong answers rather than truncated generations (§5.1). At a fixed number of typos, real-word typos are more harmful than non-word ones, clearly on GSM8K and borderline on MATH-500; on ARC they are the only kind with a detectable effect (§5.2). As corruption grows, the model flags corrupted words, and hedges, more often, yet it also misreads more words and barely re-checks its work more often (§5.3, §5.4). Finally, none of the three mitigations reliably recovers the loss (§5.5).

2 Background and Related Work

Typo robustness of LLMs. Prior studies measure final-answer accuracy under typos. Gan et al. (2024) adversarially search for up to eight character edits that derail instruction-tuned models on reasoning benchmarks, and Zhao et al. (2026) apply random keyboard edits across 18 open models and 12 languages, finding reasoning tasks among the most affected. Chai et al. (2024) attribute the damage to subword tokenization, and in concurrent work, Hu et al. (2026) link it more specifically to how many of the question’s tokens a typo breaks. Yet a typo can also form another valid word, which tokenizes normally but changes the meaning. None of these studies compares real-word and non-word typos while holding the number and positions of the typos fixed, and none looks inside the reasoning trace to see where the model goes wrong.

Reasoning traces and self-correction. A reasoning model could, in principle, notice and repair a

typo while it thinks. Concurrent work supports this: a thinking budget recovers almost all of the accuracy lost to keyboard typos (Hu et al., 2026). This evidence comes from final accuracy alone. The traces are not examined, so it remains unknown whether the model actually repairs the typos or reaches the answer despite them. Repair is also not guaranteed. On flawed input, reasoning models write much longer, redundant traces rather than resolving the problem (Fan et al., 2025), LLMs struggle to correct themselves without external feedback (Huang et al., 2024), and a written trace need not reflect the model’s actual reasoning (Turpin et al., 2023).

Simple fixes. Repairing the input before reasoning, with a separate model, with the LLM itself, or with a spell-checker, gives mixed results (Pruthi et al., 2019; Wang et al., 2024; Mu et al., 2026; Zhu et al., 2026). Spell-checking in particular can pick fluent but wrong dictionary words (Zhu et al., 2026), since a dictionary lookup cannot flag a typo that is itself a valid word (Kukich, 1992); how these fixes behave as such typos become more common has not been measured.

3 Method

3.1 Controlled typo generation

We generate corrupted versions of each question by modifying only its natural-language text. For multiple-choice questions, only the question stem is corrupted; the answer options are left unchanged. Numbers, LaTeX expressions, and LaTeX command names are protected and never modified. Candidate words must consist only of ASCII letters, and we exclude single-character words and a small set of common function words.

The generator varies two factors independently. The typo rate r determines how many eligible words are corrupted, while the real-word ratio ρ determines the target fraction of those corruptions that should result in another valid English word (see Figure 1 for an example).

Each corrupted word receives one of four keyboard edits: adjacent-key substitution (probability 0.40), deletion (0.20), insertion (0.20), or transposition (0.20). With probability 0.10, a second edit is applied to the same word.

For each question, we convert ρ into a target number of real-word typos, using probabilistic rounding when the target is not an integer. We then process the selected words in random order.

While the target has not been reached, the generator searches for valid real-word variants using the NLTK English lexicon (Bird et al., 2009). If none exists for a selected word, that word receives a non-word typo and the generator continues trying to satisfy the target with the remaining words. Any real word produced accidentally is counted toward the target. Because the selected words are not chosen based on whether a real-word variant exists, the requested ratio cannot always be reached exactly. In these cases, the generator gets as close to the target as the selected words allow. Table 1 reports the achieved ratios.

For each benchmark, we compare the clean setting with nine corrupted conditions, described as follows: each condition combines a typo rate $r \in \{25, 50, 75\}\%$ with a real-word ratio $\rho \in \{10, 40, 70\}\%$. All conditions use the same questions and the same random seed, which allows paired comparisons. The resulting datasets are available on Hugging Face (Dolevabudi/silent-tax-results).

3.2 Datasets

We use 500 questions from each of three benchmarks: GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024), and ARC-Challenge (Clark et al., 2018). GSM8K contains arithmetic word problems written mostly in words, with relatively few numbers, so each question has many words that can be corrupted. MATH-500 contains shorter competition problems with substantial LaTeX content, which our generator protects, so its questions have fewer words that are eligible for typos. ARC-Challenge contains short multiple-choice science questions, whose answer options may help the model interpret a corrupted question.

As a result, the same typo rate does not produce the same absolute number of typos across datasets: GSM8K receives the most corruption per question (Table 1).

4 Experimental Setup

Models. We use DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025), accessed through the Hugging Face API, with temperature 0.6 and top-p 0.95 (the model-card defaults, to reflect how a typical user would run it) and one response per question. For the LLM-based evaluation we use Llama-3.3-70B-Instruct (Grattafiori et al., 2024), also through the Hugging Face API, with temperature 0.

	GSM8K	MATH-500	ARC-C
questions	500	500	500
mean words per question	45.8	31.5	23.5
median words per question	43	26	20
eligible words per question	30.6	15.7	16.6
typos per question, $r=25\%$	7.6	4.0	4.2
typos per question, $r=50\%$	15.3	7.9	8.3
typos per question, $r=75\%$	22.9	11.8	12.4
achieved real-word ratio, $\rho=10\%$	0.100	0.099	0.102
achieved real-word ratio, $\rho=40\%$	0.399	0.397	0.401
achieved real-word ratio, $\rho=70\%$	0.698	0.681	0.682
total corrupted words	68,739	35,685	37,248

Table 1: Dataset and corruption statistics over the nine corrupted conditions ($r \in \{25, 50, 75\}\%$, $\rho \in \{10, 40, 70\}\%$), first 500 questions of GSM8K and ARC-Challenge and all of MATH-500. *Eligible words* are those the generator may corrupt, after protected spans, single-character words and function words are removed; the typo rate applies to them, and real-word ratios are pooled over all corrupted words.

Prompts. For math problems, we provide the question followed by the instruction: “Please reason step by step, and put your final answer within `\boxed{}`.” For multiple-choice questions, we present the four answer options with shuffled labels and ask the model to return the final answer as a boxed letter. Typos are added only to the question itself, while the instruction remains unchanged.

Scoring. We extract the final answer only from the text produced after the closing `</think>` tag. Responses that reach the token limit¹ without producing a final answer are marked as unanswered (Appendix F). GSM8K is evaluated using normalized numeric matching, MATH-500 using symbolic equivalence with `math_verify`, and ARC using exact answer-letter matching. We report accuracy over answered questions together with the number of answered responses, and check the effect of truncation with strict accuracy, which counts unanswered responses as wrong.

Statistics. Since each condition contains the same 500 questions, we use paired tests. We count the questions that change from correct to incorrect and from incorrect to correct, and compare these paired outcomes with a continuity-corrected McNemar test, which separates the effect of typos from differences in question difficulty.

Reasoning trace analysis. We analyze the reasoning traces in three ways. First, we find each

¹We cap generation at 20,000 tokens for GSM8K and 17,000 for MATH-500 and ARC-Challenge. We chose these limits mainly to remove outlier questions whose reasoning exceeds that length.

corrupted word by aligning the clean and corrupted question word by word, and classify it by whether the trace uses its original form, its corrupted form, both, or neither: silently fixed, misread, flagged, or not used (§5.3). Second, we measure self-doubt directly from the reasoning traces. We define sets of words and phrases associated with uncertainty and self-correction, and count how often they appear in each trace. Since traces differ in length, we normalize these counts by trace length. We then compare these measures across typo conditions and between correct and incorrect answers to examine whether higher uncertainty is associated with model failure (§5.4). Third, an LLM judge (Zheng et al., 2023) receives the list of corrupted words with their originals, the question as shown to the model, and the reasoning trace (its first and last 2,000 characters). Using a fixed rubric, it assigns a *self-doubt* score from 0 (a clear, straight-line solution) to 10 (pervasive uncertainty, reversals or looping), and a *repair* score from 0 (the intended meaning of the question is fully understood) to 5 (the model reasons from the wrong meaning). Higher is worse on both scales. The judge scores answered traces only.

5 Results

5.1 Effect of typos on accuracy

Figure 2 and Table 4 in the appendix show the full results. On GSM8K, answered-only accuracy drops from 92.8% on clean questions to 65.1% in the most corrupted condition ($r=75$, $\rho=70$). ARC drops from 85.2% to 74.5%, and MATH-500 from 95.2% to 86.9%. On all three datasets accuracy is lowest at the highest typo rate, although on ARC the mildest rate ($r=25$) is still on par with clean.

The paired analysis shows the same trend. In the most corrupted GSM8K condition, 145 questions change from correct to incorrect, while only 10 change from incorrect to correct, out of 482 questions answered in both conditions ($p < 10^{-6}$). ARC shows 77 correct-to-incorrect versus 25 incorrect-to-correct changes, and MATH-500 shows 46 versus 7, both statistically significant. GSM8K is affected even at the mildest typo condition, while significant effects appear only at higher typo rates for MATH-500 and ARC (Table 4). One possible explanation is how much each task depends on the wording of the question: GSM8K problems rely on the text to describe the quantities and relationships needed for the solution, ARC’s answer options may help the model recover some

information, and much of the mathematical content of MATH-500 is written in protected LaTeX.

Typos also lead to longer reasoning traces across all three datasets, while the model still produces final answers in most cases. On GSM8K strict accuracy falls from 92.8% to 62.8%; of this strict drop at $r=75$, $\rho=70$, 92% comes from answered-but-wrong responses and only 8% from truncation, and on MATH-500 truncation contributes nothing and on ARC 5%. Typos also lengthen the reasoning: on GSM8K the mean trace grows from 1,535 tokens on clean questions to 2,330 in the most corrupted condition (and up to 2,593 at $r=75$, $\rho=40$), with smaller increases on MATH-500 and ARC. This suggests that the drop in accuracy is driven mainly by incorrect answers rather than incomplete generations. In fact, wrong answers tend to have longer reasoning traces than correct ones, indicating that additional reasoning is more closely associated with difficulty than with successful recovery (Table 7).

5.2 Real-word typos are more harmful

Because typo rate and real-word ratio are varied independently, we can compare real-word and non-word typos while controlling for the total number of typos. For each dataset, we fit a logistic regression using the number of real-word and non-word typos to predict whether the final answer is correct (Figure 3a). The results show that an additional real-word typo has a stronger negative association with correctness than an additional non-word typo: the relative per-typo loss is about $1.7\times$ larger on GSM8K (95% CI 1.3–2.2), $1.3\times$ on MATH-500 (1.0–1.7, only borderline), and $5.2\times$ on ARC, but ARC’s non-word coefficient is not distinguishable from zero, so its ratio is not well defined (Table 5).

The paired comparisons support this pattern, although the effect depends on the overall typo rate. On GSM8K at a 75% typo rate, increasing the real-word share from 10% to 70% lowers accuracy by 10.4 percentage points, even though the same questions contain the same total number of typos. The difference is also significant at a 50% typo rate, but not at 25%. When corruption is mild, the model can often handle both kinds of typo, and the difference becomes more pronounced as the overall typo rate increases (Figure 3b; Table 6). ARC shows a significant real-word effect at the highest typo rate, while MATH-500 shows no significant difference at any typo rate, consistent with its smaller number of corrupted words.

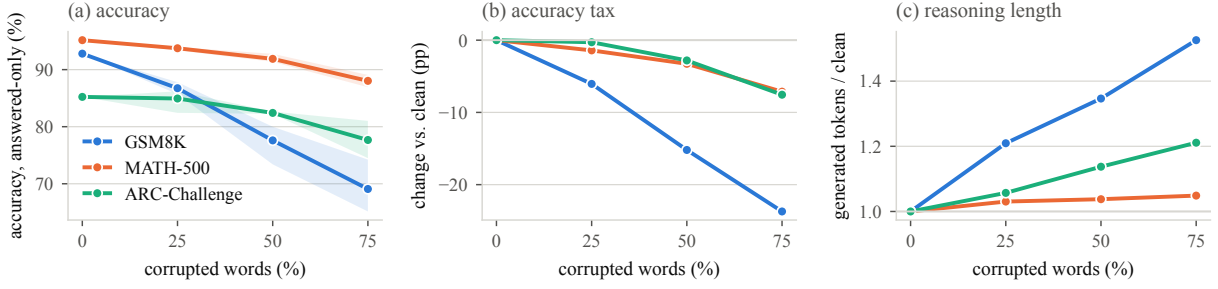


Figure 2: **The accuracy tax.** (a) answered-only accuracy against the typo rate; the line is the mean over the three real-word ratios and the band spans them. (b) the same as a change from clean, in percentage points. (c) mean generated tokens relative to clean. Accuracy falls with the typo rate on all three datasets and falls fastest on GSM8K, where the text carries the whole problem. On average, reasoning gets longer under typos, so the loss is not the model giving up early.

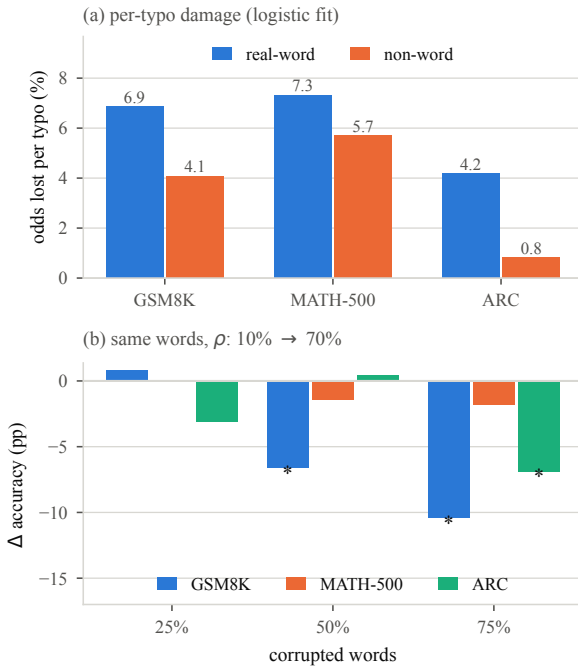


Figure 3: **The real-word effect.** (a) multiplicative loss in the odds of a correct answer per additional typo, from a logistic regression fitting real-word and non-word counts separately. (b) the controlled paired comparison: the same questions with the same number of typos, changing only the real-word share from $\rho=10$ to $\rho=70$. * marks McNemar $p < 0.05$.

Overall, real-word typos can be especially difficult for the model. Unlike non-word typos, which produce visibly unusual word forms, they result in valid words and can change the meaning of the input without creating an obvious tokenization problem.

5.3 Repair behavior: noticed but not repaired

We recover each corrupted word by comparing the clean and corrupted versions of the question, and

classify how the model handles it in the reasoning trace: **silent fix**, only the original word appears (the model recovered the intended word without mentioning the typo); **flagged**, both the original and the corrupted form appear (the model noticed and corrected the typo); **misread**, only the corrupted form appears (the model took it as the wrong word); and **not used**, neither form appears.

We originally expected real-word typos to go unnoticed more often, while non-word typos would be easier to detect and correct explicitly. The results only partly support this prediction. As the typo rate increases from 25% to 75%, silent fixing decreases on all three datasets; on GSM8K, for example, it falls from 74.0% to 55.8% at $\rho=10$. At the same time, flagging and misreading become more common: pooled over real-word ratios, misreading rises from 3.9% to 7.8% on GSM8K and from 1.4% to 2.7% on MATH-500, and less steeply on ARC, from 2.9% to 4.6% (Figure 4; Table 8).

Increasing the real-word ratio has a similar effect. Pooled across typo rates on GSM8K, silent fixing falls from 62.4% to 55.3% as ρ increases from 10 to 70, while flagging rises from 21.4% to 24.5% (peaking at 25.4% at $\rho=40$) and misreading from 5.2% to 7.5%; the same overall pattern appears on MATH-500 and ARC. Misreading is also linked to failure. We compare the misread rate in traces that end in a wrong answer with the rate in traces that end in a correct one (Table 8, *mis_w* vs. *mis_c*). On GSM8K the rate is higher in wrong-answer traces in every condition, by a factor of 1.2 to 2.1, and on ARC by 1.3 to 2.7. On MATH-500, where misreads are rare, the direction is not consistent. The share of typos that are not used stays stable within each dataset but differs across tasks, roughly 12% on GSM8K, 13% on ARC, and 20% on MATH-500,

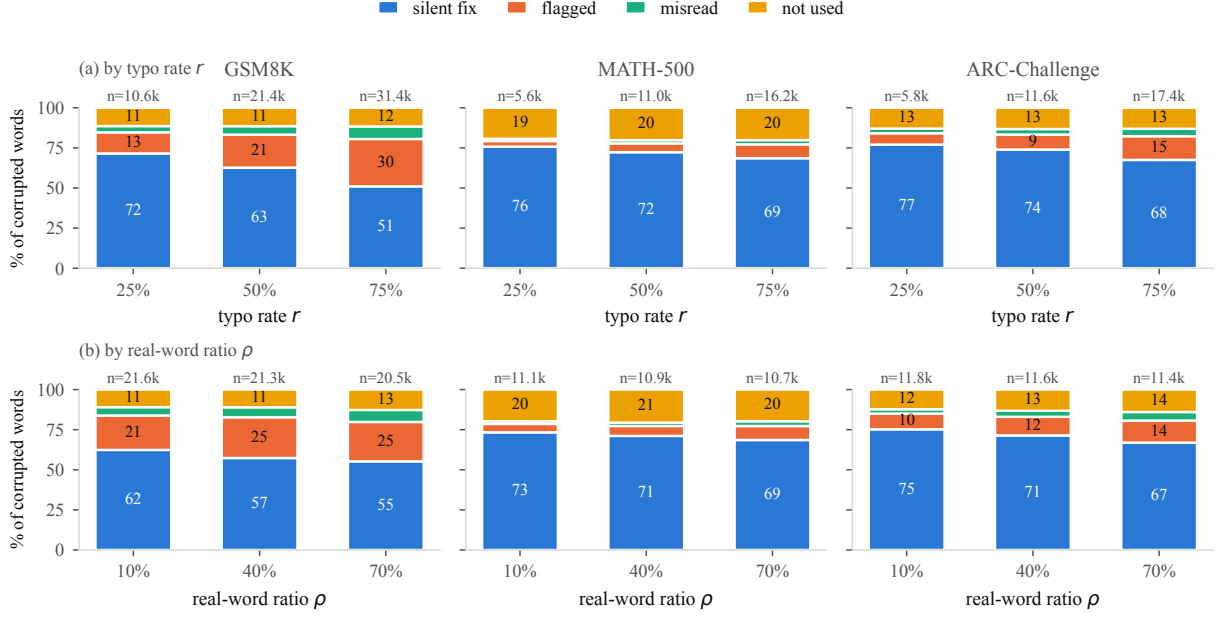


Figure 4: **Word-level repair behavior.** Each bar is 100% of the corrupted words in the conditions it pools (n above the bar, in thousands of words). The x-axis sets how many words are corrupted; the segments split those corrupted words by how the reasoning trace handles them. For example, at $r=25\%$ on GSM8K, 25% of the eligible words in each question are corrupted, and 72% of those corrupted words are silently fixed. (a) by typo rate r , pooled over real-word ratios. (b) by real-word ratio ρ , pooled over typo rates. Segments below 9% are not labeled; per-condition values are in Table 8.

possibly because MATH-500 reasoning is more symbolic and refers back to the wording of the question less often.

The judge’s repair score (§4; Table 2) tells the same story. It rises with the typo rate (on MATH-500 only at $r=75\%$), and wrong answers score higher than correct ones (over the typo conditions: 2.45 vs. 0.97 on GSM8K), linking misunderstanding of the corrupted question to failure. It also rises with the real-word ratio in every dataset and typo-rate cell, consistent with real-word typos being harder to repair, although we did not test this trend for significance.

5.4 Self-doubt

We measure self-doubt with two curated sets of expressions that we built for this study from the model’s reasoning traces. The uncertainty set (12 patterns) contains hedges such as *maybe*, *perhaps*, *possibly*, *might be*, and *not sure*; the second-guessing set (10 patterns) captures explicit reconsideration, such as *wait*, *actually*, *reconsider*, *let me recheck*, and *double-check*. The full lists and matching rules are in Appendix D. We report matches per 1,000 reasoning words.

Uncertainty increases as the input becomes more corrupted. On GSM8K, it rises from 7.0 markers

condition	GSM8K		MATH-500		ARC-C	
	repair	doubt	repair	doubt	repair	doubt
clean						
—	0.12	4.46	0.03	3.66	0.03	4.70
by typo rate r						
25%	0.88	4.86	0.35	3.90	0.97	5.12
50%	1.27	5.12	0.35	3.98	1.03	5.31
75%	1.74	5.67	0.48	4.01	1.35	5.63
by real-word ratio ρ						
10%	1.13	5.20	0.32	3.87	0.94	5.33
40%	1.32	5.29	0.39	4.03	1.10	5.28
70%	1.43	5.15	0.47	3.99	1.31	5.45
by final answer (typo configs)						
correct	0.97	4.72	0.30	3.78	0.98	5.01
wrong	2.45	6.96	1.36	5.86	1.76	6.92

Table 2: Mean LLM-judge scores over answered traces. *repair* is the repair score (0–5, higher means less of the intended meaning recovered); *doubt* is the self-doubt score (0–10, higher means more hesitation and looping). The by-final-answer rows are over the nine typo conditions only (clean traces excluded).

per 1,000 words on clean input to 12.1 at a 25% typo rate and 18.4 at 75%, with the real-word ratio held at 10%. The same trend holds on MATH-500 and ARC at the higher typo rates, although on ARC the mildest rate stays close to clean (13.0–14.2 vs. 13.6). At a fixed typo rate, a higher real-word ratio tends to add further uncertainty (Figure 5; Table 7).

Second-guessing behaves differently. Its density

risers only slightly on GSM8K, barely on MATH-500 and not at all on ARC, and stays roughly flat across typo rates and real-word ratios. The number of second-guessing expressions per trace does grow, but mostly because corrupted inputs produce longer traces. Typos thus make the model express more uncertainty without making it reconsider its reasoning much more often.

The contrast is clearer when we separate correct and incorrect answers. For the same condition, uncertainty is consistently more frequent in traces that end with a wrong answer: 1.6 to 2.1 times on GSM8K, with similar patterns on ARC and MATH-500, while second-guessing differs much less (Table 7). This relation is not unique to typos: even on clean GSM8K questions, incorrect answers contain substantially more uncertainty language than correct ones.

The judge’s self-doubt score (Table 2) rises with the typo rate on all three datasets, and typo traces score higher than clean ones everywhere. The real-word ratio, however, shows no consistent direction: pooled over typo rates, the score varies by less than 0.2 points on each dataset. The judge therefore confirms that more corruption brings more hesitation but, unlike the marker counts, does not register an additional effect of the kind of corruption.

5.5 No lightweight mitigation reliably helps

We tested the three mitigation strategies on GSM8K (Table 3).

Warn. Prefixing the prompt with “Note: the text may contain typos.” changed accuracy by only -0.7 percentage points on average across the nine typo conditions, with effects ranging from -3.2 to $+1.4$ points. This is consistent with §5.3, where the model often already showed signs of noticing that the input was corrupted: the limitation does not appear to be awareness of the typo itself.

Rewrite. Instructing the model to first rewrite the question with the typos corrected and only then solve it performed worse than the original prompt in six of the nine conditions, reducing accuracy by 2.2 percentage points on average and by up to 6.0 points; in the other three it was at most 1.3 points better. Median reasoning length also fell, from about 1,300 to about 740 tokens (median over the nine typo conditions). One possible explanation is that the rewrite step forces the model to commit to a single interpretation of the corrupted question too early; if the reconstruction is wrong,

r/ρ	no fix	warn		rewrite		spell check	
	acc	acc	Δ	acc	Δ	acc	Δ
25/10	85.7	85.9	+0.2	87.0	+1.3	87.0	+1.3
25/40	87.8	87.6	-0.2	85.3	-2.5	86.1	-1.7
25/70	86.7	85.9	-0.8	83.4	-3.4	85.5	-1.2
50/10	80.0	80.2	+0.2	80.8	+0.8	79.4	-0.6
50/40	79.4	76.2	-3.2	75.8	-3.6	75.9	-3.5
50/70	73.4	74.8	+1.4	71.1	-2.3	73.0	-0.4
75/10	74.2	71.2	-3.1	69.7	-4.5	75.7	+1.4
75/40	67.9	67.8	-0.1	67.9	+0.0	68.4	+0.5
75/70	65.1	64.8	-0.4	59.1	-6.0	62.2	-2.9
mean			-0.7		-2.2		-0.8

Table 3: GSM8K answered-only accuracy (%) under each mitigation, per condition, with the change against the unmodified prompt (*no fix*). The mean row is taken over the nine typo conditions. Δ is computed from unrounded accuracies, so it can differ by 0.1 from the difference of the values shown.

the model reasons about the wrong problem with little opportunity to recover.

Spell check. We applied an external spell checker (pyspellchecker, edit distance 2, prose words only) to the same questions before sending them to the model. A typo counts as recovered only if it is restored exactly to the original word. Recovery dropped as the share of real-word typos increased: from 55.0% at $\rho=10$ to 39.9% at $\rho=40$ and 25.8% at $\rho=70$ (pooled over all corrupted words). This lexical recovery did not translate into a consistent accuracy gain: the spell checker changed accuracy by -0.8 percentage points on average, improving three conditions and hurting six, including the hardest one (62.2% against 65.1% without it). Spell checkers detect non-word errors far better than real-word substitutions, and may even replace a non-word typo with a wrong valid word, creating the kind of error that §5.2 found to be more harmful.

6 Discussion

The most consistent finding across our analyses is that real-word typos are more harmful than non-word typos, in the per-typo regression, the paired comparisons, and the spell-checker results. One might expect the opposite, since non-word typos disrupt tokenization. Our results suggest that a non-word typo is visibly unusual and signals that something is wrong, whereas a real-word typo is fluent, tokenized normally, and may fit the sentence well enough that its wrong meaning is taken at face value. The same property makes it hard for

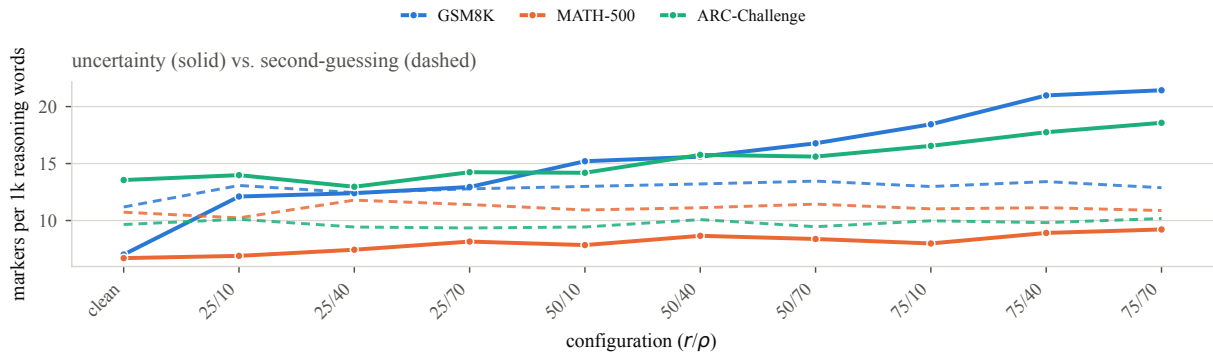


Figure 5: **Self-doubt under typos.** Marker density per 1,000 reasoning words across the grid. Uncertainty markers (solid) rise with corruption on all three datasets, most steeply on GSM8K, while second-guessing markers (dashed) stay roughly flat: typos make the model express more doubt without making it verify more. The same densities split by final answer are in Table 7.

dictionary-based correction, which can only act on forms that are not valid words.

The trace analysis points to a related conclusion. The model often notices that the input is unusual: uncertainty language increases, corrupted words are flagged more often, and traces become longer. This additional effort does not reliably lead to recovery, which also explains why the mitigations had little effect: a warning adds information the model often already has, while a forced rewrite commits it early to a single, possibly wrong, interpretation. A more promising direction may be to keep multiple interpretations open during reasoning and resolve them later using the structure of the problem.

7 Conclusion

Typos have a clear and consistent effect on reasoning performance. On GSM8K, accuracy dropped by as much as 27.7 percentage points, with larger declines as the corruption rate increased, and the strongest effect came from typos that turned one valid English word into another. The main problem is not that the model fails to notice corrupted input: it often recognizes that something is wrong, but cannot reliably recover the intended meaning. Its traces show more uncertainty rather than more verification, and neither prompting strategies nor dictionary-based correction consistently improved performance. Robustness to everyday typing errors should therefore be treated as a capability to be developed explicitly, rather than assumed from strong benchmark performance.

All code is available at https://github.com/dolev98/NLP_project, and the typo datasets, raw generation logs and judge scores are

hosted on the Hugging Face Hub ([dolevabudi/silent-tax-results](https://huggingface.co/dolevabudi/silent-tax-results)). All reported results can be reproduced by running `download_data.py` followed by `run_all.py`.

Limitations

One model, one sample. All headline results come from a single 7B reasoning model at one decoding setting with $n=1$ sample per question. Our plan called for several samples per question, but the API budget did not permit it, so we cannot separate typo effects from decoding variance at the level of an individual question. The overall effects are much larger than expected sampling noise (McNemar $p < 10^{-6}$ on $n=482$ paired items), but generalization to larger reasoning models is untested.

Easier multiple-choice dataset. Our multiple-choice results come from ARC-Challenge, which is easier than GPQA-Diamond (Rein et al., 2024), the science dataset we originally planned to use. On clean GPQA-Diamond the model reached only about 52% accuracy against a 25% chance level, which leaves little room to measure further drops before changes reflect guessing rather than reasoning. Because ARC-Challenge is easier and its answer options may help the model recover information, our multiple-choice results may underestimate the effect of typos on harder science questions.

Scope of the mitigation study. We tested the mitigations only on GSM8K, because each required additional paid API calls; we chose GSM8K because it showed the clearest drop under typos, making it the most informative dataset for testing whether a mitigation recovers performance.

AI Disclosure and Reflection

What we used AI for. We used GitHub Copilot with Claude primarily to support coding tasks during the project. It helped us build the typogeneration and inference pipelines, write analysis scripts, generate matplotlib code for figures, and improve the wording of some parts of the paper. All methodological and experimental decisions, including the two-axis corruption grid, typo-type weights, answer-extraction rules, and judge rubric, were made by us. The analysis scripts were written with Claude’s assistance based on these decisions, and all reported results were generated from the raw generation logs using those scripts.

Reflection. AI tools were especially useful for speeding up repetitive coding and analysis tasks and for keeping the implementation consistent across experiments. However, generated code and outputs still required careful checking, particularly for numerical results. We therefore verified reported results against the raw data and used a consistent scoring procedure throughout the analysis. Overall, the project highlighted the importance of reproducibility and independent verification when AI tools are used in research.

References

- Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O’Reilly Media, Sebastopol, CA.
- Yekun Chai, Yewei Fang, Qiwei Peng, and Xuhong Li. 2024. [Tokenization falling short: On subword robustness in large language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1582–1599, Miami, Florida, USA. Association for Computational Linguistics.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. [Think you have solved question answering? Try ARC, the AI2 reasoning challenge](#). *arXiv preprint arXiv:1803.05457*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, et al. 2021. [Training verifiers to solve math word problems](#). *arXiv preprint arXiv:2110.14168*.
- Chenrui Fan, Ming Li, Lichao Sun, and Tianyi Zhou. 2025. [Missing premise exacerbates overthinking: Are reasoning models losing critical thinking skill?](#) In *Second Conference on Language Modeling*.
- Esther Gan, Yiran Zhao, Liying Cheng, Yancan Mao, Anirudh Goyal, Kenji Kawaguchi, Min-Yen Kan, and Michael Shieh. 2024. [Reasoning robustness of LLMs to adversarial typographical errors](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 10449–10459, Miami, Florida, USA. Association for Computational Linguistics.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. 2024. [The Llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. [DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning](#). *Nature*, 645(8081):633–638.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. [Measuring mathematical problem solving with the MATH dataset](#). In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- Zizhao Hu, Nathan Elijah Segura, Mohammad Rostami, and Jesse Thomason. 2026. [Should we type or talk to LLM agents? A comprehensive study of voice and keyboard input perturbations](#). *arXiv preprint arXiv:2608.03970*.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. [Large language models cannot self-correct reasoning yet](#). In *The Twelfth International Conference on Learning Representations*.
- Karen Kukich. 1992. [Techniques for automatically correcting words in text](#). *ACM Computing Surveys*, 24(4):377–439.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. [Let’s verify step by step](#). In *The Twelfth International Conference on Learning Representations*.
- Lin Mu, Guowei Chu, Li Ni, Lei Sang, and Yiwen Zhang. 2026. [Robustness of prompting: Enhancing robustness of large language models against prompt attacks](#). *IEEE Transactions on Artificial Intelligence*. To appear; arXiv:2506.03627.
- Danish Pruthi, Bhuwan Dhingra, and Zachary C. Lipton. 2019. [Combating adversarial misspellings with robust word recognition](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5582–5591, Florence, Italy. Association for Computational Linguistics.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. [GPQA: A graduate-level Google-proof Q&A benchmark](#). In *First Conference on Language Modeling*.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. [Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Bin Wang, Chengwei Wei, Zhengyuan Liu, Geyu Lin, and Nancy F. Chen. 2024. [Resilience of large language models for noisy instructions](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 11939–11950, Miami, Florida, USA. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Raoyuan Zhao, Yihong Liu, Lena Altinger, Hinrich Schütze, and Michael A. Hedderich. 2026. [Evaluating robustness of large language models against multilingual typographical errors](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16059–16078, San Diego, California, United States. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging LLM-as-a-judge with MT-Bench and Chatbot Arena](#). In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 36, pages 46595–46623.

Jiaqian Zhu, Yang Zhang, Junhua Ding, and Xiaowei Yu. 2026. [Lexical perturbations disrupt LLM reasoning: An empirical study of attention diversion](#). In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. To appear; arXiv:2608.22140.

A Full accuracy grid

Table 4 gives accuracy, answered questions, mean generated tokens and paired flips against clean for every condition on every dataset.

B Per-typo logistic fit

Table 5 gives the per-typo logistic fit behind §5.2.

C Paired real-word contrasts

Table 6 gives the paired real-word contrasts behind §5.2, at every typo rate and every pair of real-word ratios.

D Reasoning length and self-doubt markers

Self-doubt marker sets. The sets were compiled from expressions of hedging and of explicit reconsideration found in the model’s reasoning traces, together with the examples in our project plan. Expressions that name the typo itself (e.g., *typo*, *misspelled*) were left out, since noticing a typo is part of repair behaviour (§5.3) rather than self-doubt. Because the sets come from this model’s traces, they may miss hedges that other models use; every condition is scored with the same sets. Each pattern is matched in the reasoning trace as a whole word or phrase, ignoring case. Within a set, overlapping matches (e.g., *but wait* and *wait*) count as one expression. Densities are matches per 1,000 words of reasoning.

Uncertainty (12 patterns): *not sure* (also *unsure*, *not certain*, *not entirely sure*, *not totally sure*); *maybe*; *perhaps*; *possibly*; *might be*; *could be*; *I guess*; *I assume* (also *I’ll assume*); *presumably*; *or maybe* (also *alternatively*, *then again*); *unclear* (also *not clear*, *isn’t clear*, *hard to tell*, *ambiguous*, *confused*, *confusing*); *not entirely*.

Second-guessing (10 patterns): *wait*; *hmm*; *actually*; *reconsider*; *but wait*; *wait, no* (also *actually*, *no, no, wait*); *hold on*; *on second thought*; *let me recheck* (also *let me re-read*, *let me verify*, *let me redo*, *let me double-check*); *double-check* (also *double check*, *double-checking*, *double-checked*).

Table 7 gives reasoning length and marker densities for every condition, overall and split by final answer.

E Word-level repair behaviour

Table 8 classifies every corrupted word in every condition by how the reasoning trace handles it (§5.3).

F Responses that reach the token limit

Table 9 counts the responses that reach the generation cap in each condition.

condition	GSM8K						MATH-500						ARC-Challenge					
	acc	n_a	tok	r→w	w→r	p	acc	n_a	tok	r→w	w→r	p	acc	n_a	tok	r→w	w→r	p
clean	92.8	500	1535	—	—	—	95.2	496	2927	—	—	—	85.2	494	1167	—	—	—
typo25_real10	85.7	497	1913	47	12	9.6e-06	93.9	495	2945	12	7	0.359	86.2	492	1218	26	30	0.688
typo25_real40	87.8	500	1814	39	14	9.8e-04	93.4	498	3111	15	9	0.307	86.2	493	1243	27	32	0.603
typo25_real70	86.7	497	1843	43	12	5.2e-05	93.9	494	2992	15	9	0.307	82.4	495	1239	41	26	0.087
typo50_real10	80.0	500	2024	78	14	<1e-6	92.8	498	2939	18	9	0.124	82.2	490	1296	42	28	0.120
typo50_real40	79.4	500	2084	83	16	<1e-6	91.4	498	3085	22	6	0.005	82.4	493	1359	42	25	0.051
typo50_real70	73.4	500	2094	108	11	<1e-6	91.5	496	3087	25	9	0.010	82.7	490	1325	44	30	0.131
typo75_real10	74.2	489	2104	101	8	<1e-6	89.1	494	2945	34	5	7.3e-06	81.0	490	1383	56	35	0.036
typo75_real40	67.9	495	2593	136	13	<1e-6	88.1	496	3125	39	7	4.9e-06	77.5	489	1408	65	28	1.9e-04
typo75_real70	65.1	482	2330	145	10	<1e-6	86.9	497	3137	46	7	<1e-6	74.5	490	1447	77	25	<1e-6

Table 4: Every condition on every dataset. *acc* is answered-only accuracy (%), *n_a* the number of questions that produced a final answer, *tok* the mean generated tokens. r→w and w→r are paired right-to-wrong and wrong-to-right flips against the clean condition over jointly answered questions, with the continuity-corrected McNemar p-value (uncorrected for multiple comparisons).

	GSM8K	MATH-500	ARC-C
traces in the fit, n	4460	4466	4422
intercept	+2.136	+3.019	+1.686
coefficient, real-word	-0.0712	-0.0761	-0.0428
coefficient, non-word	-0.0416	-0.0588	-0.0081
odds multiplier, real-word	0.931	0.927	0.958
odds multiplier, non-word	0.959	0.943	0.992
odds lost per real-word typo	6.9%	7.3%	4.2%
odds lost per non-word typo	4.1%	5.7%	0.8%
relative loss, real / non-word	1.7×	1.3×	5.2×
95% CI of the relative loss (bootstrap over questions)	[1.3, 2.2]	[1.0, 1.7]	n/a [†]
non-word coefficient, 95% CI	[-0.0541, -0.0298]	[-0.0792, -0.0398]	[-0.0272, +0.0118]

Table 5: Per-typo logistic fit behind §5.2. One regression per dataset, $\text{correct} \sim \text{num_real} + \text{num_nonword}$, fitted over all answered typo traces pooled across the nine conditions. The predictors are the counts of each kind of typo actually applied to that question. An odds multiplier is $\exp(\text{coefficient})$: the factor by which one additional typo of that kind multiplies the odds of a correct answer. [†] On ARC the non-word coefficient’s CI includes 0, so the ratio has no finite interval; the difference of the two coefficients is -0.035 (95% CI -0.057 to -0.012 , $p=0.003$).

<i>r</i>	ρ	GSM8K			MATH-500			ARC-Challenge		
		Δacc	r/w	p	Δacc	r/w	p	Δacc	r/w	p
25	10→70	+0.8	31/35	0.712	+0.0	10/10	1.000	-3.1	45/30	0.106
25	10→40	+2.0	25/35	0.245	+0.0	10/10	1.000	+0.2	32/33	1.000
25	40→70	-1.2	31/25	0.504	-0.2	13/12	1.000	-4.1*	41/21	0.016
50	10→70	-6.6*	74/41	0.003	-1.4	19/12	0.281	+0.4	41/43	0.913
50	10→40	-0.6	45/42	0.830	-1.4	17/10	0.248	+0.2	36/37	1.000
50	40→70	-6.0*	69/39	0.005	-0.2	13/12	1.000	+0.2	37/38	1.000
75	10→70	-10.4*	92/43	3.6e-05	-1.8	28/19	0.243	-6.9*	61/28	6.9e-04
75	10→40	-5.6*	85/58	0.030	-0.2	20/19	1.000	-4.0	52/33	0.051
75	40→70	-3.6	72/55	0.156	-1.6	25/17	0.280	-3.1	60/45	0.172

Table 6: Controlled paired contrast behind §5.2, at every typo rate and every pair of real-word ratios. Within a fixed *r* the same words of the same questions are corrupted in the same positions, so only the *kind* of corruption changes. Δacc is the accuracy of the higher- ρ variant minus the lower, over questions answered in both; *r/w* is right-to-wrong against wrong-to-right flips; *p* is a continuity-corrected McNemar test and * marks $p<0.05$ before multiple-comparison correction.

condition	GSM8K						MATH-500						ARC-Challenge					
	tok	tok_c	tok_w	uncert.	2nd-g.	u.w/c	tok	tok_c	tok_w	uncert.	2nd-g.	u.w/c	tok	tok_c	tok_w	uncert.	2nd-g.	u.w/c
clean	1535	1410	3156	7.0	11.2	1.59	2927	2803	5352	6.7	10.7	1.70	1167	1018	2026	13.6	9.7	1.71
typo25_real10	1913	1509	4335	12.1	13.1	1.83	2945	2826	4783	6.9	10.2	1.61	1218	1118	1837	14.0	10.1	1.57
typo25_real40	1814	1558	3661	12.4	12.5	1.91	3111	2900	6075	7.4	11.8	1.52	1243	1111	2069	13.0	9.4	1.72
typo25_real70	1843	1537	3843	12.9	12.8	2.12	2992	2859	5047	8.2	11.4	1.66	1239	1073	2015	14.2	9.3	1.91
typo50_real10	2024	1549	3922	15.2	13.0	1.87	2939	2763	5191	7.8	10.9	1.97	1296	1182	1828	14.2	9.4	1.51
typo50_real40	2084	1626	3849	15.6	13.2	1.64	3085	2839	5694	8.7	11.1	1.96	1359	1214	2037	15.8	10.1	1.91
typo50_real70	2094	1492	3755	16.8	13.5	1.66	3087	2854	5600	8.4	11.4	1.90	1325	1176	2033	15.6	9.5	1.66
typo75_real10	2104	1598	3559	18.4	13.0	1.60	2945	2787	4236	8.0	11.0	2.05	1383	1236	2012	16.6	10.0	1.48
typo75_real40	2593	1767	4340	21.0	13.4	1.61	3125	2881	4932	8.9	11.1	2.06	1408	1260	1919	17.8	9.8	1.40
typo75_real70	2330	1627	3643	21.4	12.9	1.64	3137	2911	4640	9.2	10.9	1.75	1447	1260	1995	18.6	10.2	1.47

Table 7: Supporting numbers for §5.1 and §5.4. *tok* is mean generated tokens over all answered traces; *tok_c* and *tok_w* are the same mean over traces that ended in a correct and in a wrong answer. *uncert.* and *2nd-g.* are uncertainty and second-guessing markers per 1,000 reasoning words, and *u.w/c* is uncertainty density in wrong-answer traces divided by correct-answer traces. Traces generally lengthen with the typo rate (MATH-500 at $\rho=10$ and $\rho=40$ is flat), a wrong answer is preceded by a longer trace than a correct one in every condition, and uncertainty climbs while second-guessing stays roughly flat.

<i>r</i>	ρ	GSM8K						MATH-500						ARC-Challenge					
		silent	flag	mis.	unused	mis_c	mis_w	silent	flag	mis.	unused	mis_c	mis_w	silent	flag	mis.	unused	mis_c	mis_w
25	10	74.0	11.3	2.9	11.9	2.5	5.1	77.7	1.7	1.2	19.3	1.3	0.4	80.8	5.2	1.9	12.0	1.6	3.8
25	40	72.4	12.9	4.1	10.5	3.8	6.7	74.5	3.3	1.5	20.7	1.2	3.4	76.9	7.0	2.6	13.5	2.1	5.1
25	70	68.1	15.4	4.7	11.8	4.2	7.8	75.0	5.6	1.5	17.9	1.3	3.4	73.3	9.1	4.1	13.4	3.7	5.7
50	10	66.3	18.4	4.6	10.7	3.7	7.6	75.1	3.6	1.4	19.9	1.2	2.5	77.5	7.6	2.4	12.5	1.8	4.9
50	40	62.6	21.4	5.0	11.0	4.8	5.8	71.2	6.0	2.1	20.7	2.0	2.7	74.0	9.6	3.4	13.1	3.0	4.7
50	70	59.0	22.0	6.3	12.7	5.8	7.4	70.5	6.9	2.6	20.0	2.1	5.3	70.3	10.9	4.7	14.1	4.5	5.6
75	10	55.8	26.9	6.5	10.8	5.8	8.2	70.6	7.6	2.0	19.8	1.6	3.4	71.9	12.7	3.2	12.1	2.8	5.0
75	40	48.8	32.3	7.8	11.2	6.1	10.8	69.9	7.3	2.3	20.5	1.6	4.8	67.9	14.5	4.5	13.1	4.1	5.8
75	70	48.3	29.4	9.2	13.0	7.8	11.5	65.1	11.0	3.7	20.2	3.1	5.7	62.5	17.2	6.2	14.0	5.3	8.9

Table 8: Every corrupted word in every condition, classified by how the reasoning trace handles it, as a percentage of the corrupted words in that condition. *silent* / *flag* / *mis.* / *unused* are the four categories of §5.3. *mis_c* and *mis_w* are the misread rate among questions the model answered correctly and among those it answered wrongly, which is the comparison behind the claim that misreading is associated with failure. Figure 4 shows the same data pooled along each axis. Words whose corrupted form also appears in the model’s trace on the clean question (about 3% overall) are left out.

condition	GSM8K	MATH-500	ARC-Challenge
clean	0	4	6
typo25_real10	3	5	8
typo25_real40	0	2	7
typo25_real70	3	6	5
typo50_real10	0	2	10
typo50_real40	0	2	7
typo50_real70	0	4	10
typo75_real10	11	6	10
typo75_real40	5	4	10
typo75_real70	17	3	10
all ten	39 (0.8%)	38 (0.8%)	83 (1.7%)

Table 9: Responses that reach the generation cap (20,000 tokens for GSM8K, 17,000 for MATH-500 and ARC-Challenge), out of 500 questions per condition; the last row is out of 5,000. None of them contains a final answer, so all are counted as unanswered. At most 17 of 500 (3.4%) reach the cap in any condition. The remaining unanswered responses in Table 4 stopped before the cap without an extractable answer.